

RideFlow Developer Onboarding and Handoff Handbook

Project: RideFlow

Repository: <https://github.com/raiven427/ride-flow>

Audience: The developer taking over implementation, deployment, and maintenance

Author: Manus AI

Handoff status: Portable full-stack foundation; not yet a public production ride-dispatch service

1. Welcome and first principle

RideFlow is a premium ride-sharing application designed around upfront KSh fares, driver choice, transparent 5% platform commission, safety tools, customer preferences, driver onboarding, and an admin Control Room. The code has been converted from a hosted demo into a portable React/Express/tRPC/Drizzle application so a new developer can run it on Ubuntu and connect independent infrastructure.

The most important rule for taking over the project is to separate **what the repository already implements** from **what external services must still be configured** and **what production features still need to be built**. The repository contains the code and migrations. It does not automatically provide a production database, identity provider, object-storage bucket, payment merchant accounts, realtime dispatch service, monitoring, legal review, or customer support operation.

Never place passwords, access tokens, database credentials, payment keys, webhook secrets, identity documents, or private storage URLs in GitHub, screenshots, source code, or this handbook.

2. Handoff goals for the first week

The first week should establish a reproducible local environment, verify the current test suite, connect a disposable database, run migrations, inspect the protected flows, and document any environment-specific failure. Do not begin by changing several infrastructure layers at once.

Day	Outcome
1	Clone the private repository, install dependencies, run <code>pnpm check</code> , <code>pnpm test</code> , and <code>pnpm build</code>
2	Connect a disposable MySQL/TiDB database and apply migrations
3	Configure an OIDC development application and verify sign-in/logout
4	Configure private S3-compatible storage and test a non-sensitive upload
5	Review fare, ledger, admin, presence, and activity procedures with the owner
6	Test Stripe/Daraja sandbox contracts and callback idempotency
7	Write a gap list and implementation plan for dispatch, GPS, trips, support, and production controls

3. Local Ubuntu setup

Install Git, Node.js, pnpm, and basic tools on Ubuntu 22.04 or 24.04. Then clone the private repository and use the package manager version declared by `package.json`.

```
sudo apt update
sudo apt install -y git curl unzip
curl -fsSL https://get.pnpm.io/install.sh | sh -
source ~/.bashrc
git clone https://github.com/raiven427/ride-flow.git
cd ride-flow
pnpm install --frozen-lockfile
pnpm check
pnpm test
pnpm build
```

The current scripts are:

Command	Purpose
<code>pnpm dev</code>	Starts the development server with TypeScript watch mode
<code>pnpm check</code>	Runs TypeScript checking without emitting files
<code>pnpm test</code>	Runs the Vitest suite
<code>pnpm build</code>	Builds the Vite frontend and bundles the server
<code>pnpm start</code>	Starts the production bundle
<code>pnpm db:push</code>	Generates and applies migrations in the configured environment; use carefully

Never run destructive database commands against an unknown production database. For schema changes, update `drizzle/schema.ts`, generate a migration, inspect the SQL, test it on a disposable database, back up production, and apply the approved migration.

4. Architecture map

The browser renders the React frontend. React Query and the typed tRPC client call the Express server through `/api/trpc`. The server builds request context, authenticates the session, authorizes protected/admin procedures, calls database helpers, and returns typed data. Drizzle maps the TypeScript schema to MySQL/TiDB. S3 stores file bytes. Provider adapters handle OIDC, notifications, Stripe, and Daraja.

Browser

| React 19 + Tailwind 4 + tRPC client



Express server

- OIDC callback and session cookies
- /api/trpc procedures
- /healthz
- Stripe webhook route
- Daraja callback route
-
- Drizzle ORM — MySQL/TiDB database
- S3 adapter — private object storage
- OIDC provider — login and identity
- Stripe — test/live payment events
- Safaricom Daraja — STK Push callbacks
- Notification webhook — owner alerts

5. Repository map

Frontend

File or directory	Responsibility
<code>client/src/main.tsx</code>	React bootstrap, tRPC/Query providers, global unauthorized handling
<code>client/src/App.tsx</code>	Theme, route switch, error boundary, top-level application shell
<code>client/src/pages/Home.tsx</code>	Main customer, driver, profile, booking, admin operations, and signup experience
<code>client/src/index.css</code>	Editorial Transit design system, layout, responsive rules, animations, operations and payment styles
<code>client/src/const.ts</code>	Secure browser login redirect helper
<code>client/src/lib/trpc.ts</code>	Typed tRPC client binding
<code>client/src/_core/hooks/useAuth.ts</code>	Current user, login, logout, loading, and error state
<code>client/src/components/ui/*</code>	Reusable UI primitives
<code>client/src/components/DashboardLayout.tsx</code>	Reusable internal dashboard layout
<code>client/src/components/Map.tsx</code>	Map provider boundary for future directions, places, and live tracking

Backend

File	Responsibility
<code>server/_core/index.ts</code>	Express startup, static frontend, API mounting, health route, host/port binding
<code>server/_core/env.ts</code>	Reads and validates environment-driven configuration
<code>server/_core/context.ts</code>	Creates tRPC context and current authenticated user
<code>server/_core/trpc.ts</code>	Public, protected, and admin procedure middleware
<code>server/_core/oauth.ts</code>	Provider-neutral OIDC/OAuth callback and token flow
<code>server/_core/cookies.ts</code>	Secure session-cookie behavior
<code>server/_core/notification.ts</code>	Notification provider boundary
<code>server/db.ts</code>	Database connection, user upsert, profiles, fares, ledger, presence, events, admin settings
<code>server/routers.ts</code>	tRPC API contract used by the frontend
<code>server/fare.ts</code>	Server-side distance/time/minimum/safety calculation and 5% commission split
<code>server/payments.ts</code>	Stripe and Daraja configuration, readiness checks, webhook/callback scaffolding
<code>server/storage.ts</code>	S3-compatible upload and signed-URL operations

Database and tests

Location	Responsibility
<code>drizzle/schema.ts</code>	Database source of truth
<code>drizzle/relations.ts</code>	Drizzle relations
<code>drizzle/0000*.sql</code>	Ordered migration history
<code>drizzle/meta/*</code>	Migration snapshots and journal
<code>server/*.test.ts</code>	Authorization, storage, fare, payment, activity, admin, and auth regression coverage
<code>docs/deployment-environment.template</code>	Safe names-only environment template
<code>todo.md</code>	Project history and remaining work

6. Current database model

The existing schema includes the following tables.

Table	Purpose
<code>users</code>	Identity, email, role, creation/update/sign-in timestamps
<code>rideflow_profiles</code>	Customer/driver role, phone, vehicle, insurance, photo reference, driver verification status
<code>rideflow_files</code>	Private file metadata, purpose, storage key, review status, owner
<code>rideflow_admin_settings</code>	Singleton owner email, notification email, ownership configuration
<code>rideflow_fare_rules</code>	City pricing, minimum fare, safety fee, platform commission basis points
<code>rideflow_fare_quotes</code>	Route labels, distance/time, fare components, rider total, driver earnings, quote status and expiry
<code>rideflow_ledger_entries</code>	Rider charge, driver earning, platform commission, refund, tip, and payout entries
<code>rideflow_presence</code>	Current status, current view, last-seen heartbeat
<code>rideflow_activity_events</code>	Append-only sign-ins, quotes, uploads, admin changes, and operational summaries

The commission default is 500 basis points, equivalent to 5%. The server calculates the commission and driver earnings. The browser must not be trusted to calculate or authorize a payment amount.

Production schema still required

A real dispatch operation needs additional schema for trips, driver availability, driver locations, assignments, cancellations, scheduled rides, multi-stop routes, messages, safety contacts, trip shares, ratings, favorites, payment transactions, refunds, disputes, payouts, and support cases. Do not add these as informal JSON blobs when they require authorization, reporting, or reconciliation. Design each table, migration, procedure, and test together.

7. Authentication and administrator operation

The identity provider owns the password and primary authentication. RideFlow stores the authenticated user identity and role. The admin role is separate from the identity-provider

password. The initial admin owner is configured through the application's admin settings, and verified ownership transfer is supported.

A new developer should test three identities: a customer, a driver, and an administrator. Confirm that ordinary users cannot call admin procedures directly even if the frontend is manipulated. Confirm that the old administrator loses access after a transfer. Rotate any password shared previously and do not reuse it for production.

Required OIDC settings include issuer, token, userinfo, client ID, client secret, browser authorize URL, browser client ID where applicable, scope, exact callback URL, and secure cookie configuration. Login failures should be debugged using timestamps and server logs, never by printing tokens.

8. File storage and document handling

RideFlow stores file bytes in private S3-compatible storage and file metadata in `rideflow_files`. Supported purposes include profile photo, driver license, insurance, vehicle document, and lost item. The server validates MIME type and size, uses a storage key, and returns short-lived signed access where appropriate.

The web server's local filesystem is not durable. A new developer must verify that profile images and driver documents survive server restart because the bytes are in object storage, not because the host happened to keep local files. Test with synthetic documents only and establish a deletion/retention policy before collecting real identity documents.

9. Payments and the 5% commission

Stripe

Use Stripe test mode first. Configure the server secret, browser publishable key when required, webhook secret, and Connect client ID. The webhook handler must verify the raw request signature and be idempotent. Test successful payment, decline, duplicate event, invalid signature, timeout, refund, and Connect readiness. Do not mark a ride paid from a browser response alone.

M-Pesa Daraja

Use the Daraja sandbox first. Configure consumer key/secret, environment, shortcode, passkey, callback base URL, and any required initiator/security values. STK Push is asynchronous: the initial response acknowledges request acceptance; the callback resolves success or failure. Store provider request IDs in a pending transaction, match callback identifiers, and make repeated callbacks harmless.

Ledger behavior

For a completed ride, the application should be able to reconcile the rider charge, driver earnings, and platform commission. Financial records should be append-only or compensated by explicit refund/reversal entries. Never update a historical commission amount merely because a fare rule later changes.

10. Admin operations and activity monitoring

The operations dashboard reads presence and activity data through an admin-only procedure. Authenticated users send a heartbeat with status and current view. A recent heartbeat is treated as online; an old heartbeat becomes stale/offline. The activity feed records meaningful events such as sign-in, fare quote creation, upload, notification-setting change, and admin ownership transfer.

Implement retention before production. Keep detailed activity for an approved period, retain security-sensitive admin events longer where required, avoid passwords/payment credentials/raw documents in metadata, and never place every GPS point in the general activity feed. Cleanup should run as a deterministic, bounded database job with metrics and failure alerts.

11. Deployment checklist

1. Create the private GitHub deployment source and protect the main branch.
2. Create a MySQL/TiDB database and least-privilege application user.
3. Apply migrations in order after reviewing generated SQL.
4. Create a private S3-compatible bucket and restricted access key.
5. Create an OIDC application and configure the exact production callback URL.
6. Create the Node/Express web service and use the repository build/start commands.
7. Add production secrets only through the host secret manager.
8. Deploy and verify `/healthz`.
9. Sign in as customer, driver, and admin test accounts.
10. Test quote creation, upload, admin authorization, and activity writes.
11. Configure Stripe and Daraja sandbox callbacks and run the failure matrix.
12. Configure backups, monitoring, alerts, DNS, HTTPS, and rollback procedures.
13. Build realtime dispatch and safety workflows before a public pilot.

14. Complete legal, privacy, driver verification, support, refund, and incident-readiness reviews.

12. Safe maintenance workflow

Before changing code, create a branch or checkpoint and add the requirement to `todo.md`. Inspect existing components and procedures before creating replacements. Update the schema first for database work. Add or update Vitest tests. Run `pnpm check`, `pnpm test`, and `pnpm build`. Test the browser success and failure states. Review logs for server and network errors. Commit only reviewed files and push to the private GitHub repository.

For a new feature, use this sequence:

Requirement

- domain model and authorization decision
- schema and migration if needed
- database helper
- protected/public/admin tRPC procedure
- frontend state, loading, empty, error, and success UI
- Vitest regression tests
- browser verification
- documentation and deployment notes

For a production incident, preserve the timestamp, affected environment, request ID, provider transaction ID, and relevant logs. Do not expose secrets while debugging. Roll back the application through version history if necessary; repair database state through a reviewed migration or compensating transaction.

13. Definition of a successful handoff

The new developer should be able to clone the private repository, install dependencies, run all checks, connect a disposable database, apply migrations, configure a test identity provider, upload a synthetic image to private storage, create a server-side fare quote, inspect the 5% ledger split, load the admin operations dashboard, and explain which features remain prototypes.

The handoff is not complete merely because the frontend looks polished. A production-ready RideFlow service also needs realtime dispatch, trip state, secure location sharing, payment reconciliation, refunds, support, monitoring, backups, retention enforcement, and the legal/operational controls required for transport services.

References

- [1]: <https://github.com/raiven427/ride-flow> RideFlow private source repository.
- [2]: <https://docs.stripe.com/webhooks> Stripe, "Receive Stripe events in your webhook endpoint."
- [3]: <https://developer.safaricom.co.ke/apis/MpesaExpressSimulate> Safaricom Daraja, "M-Pesa Express Simulate."
- [4]: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API MDN Web Docs, "WebSocket API."